Version Control System

AIM: *To explore Version Control System (VCS) tools, specifically Git, for effective source code management.*

Objectives:

- *Understand the concepts of VCS: local and remote repositories.*
- *Learn basic Git commands for repository management.*
- *Understand how to commit, push, pull, and clone repositories.*

Hardware Requirements: *Computer with internet connection*

Software Requirements:

- *Git for Windows*
- *Visual Studio Code (VS Code)*
- *GitHub account*

Procedure :

1) *Install Git and VS Code.*
2) *Initiate repository*

```
PS D:\VII Sem\DT Lab\2> git init
Initialized empty Git repository in D:/VII Sem/DT Lab/2/.git/
```

3) *Add files using*

```
PS D:\VII Sem\DT Lab\2> git add .
```

4) *Commit changes using*

```
PS D:\VII Sem\DT Lab\2> git commit -m "Initial commit"
```

5) *Create a repository on GitHub.*
6) *Link local repository to remote using*

```
PS D:\VII Sem\DT Lab\2> git remote add origin "https://github.com/surajreddy771"
```

7) *Push code to remote repository*

```
PS D:\VII Sem\DT Lab\2> git push origin main
```

8) *Test pulling changes from remote using*

```
PS D:\VII Sem\DT Lab\2> git pull
```

Execution:

1. *Open Git Bash and VS Code.*
2. *Create a folder for the project and initialize it as a Git repository.*
3. *Create a sample file (e.g., hello.txt) and write some text inside.*
4. *Use Git commands to add, commit, push, and pull files.*
5. *Verify that the local repository is synchronized with the GitHub repository.*

Program Code:

bash

```
>git init
>git add .
>git commit -m "Initial commit"
>git remote add origin
>git push -u origin master
```

Version Control Systems for Source Code Management:

A Version Control System (VCS) is a tool that tracks changes in source code over time, enabling multiple people to collaborate efficiently. It helps in:

- *Storing different versions of code.*
- *Rolling back to earlier versions if needed.*
- *Managing contributions from multiple developers.*

Types of Version Control Systems:

1. *Local VCS – Stores versions in a local database (e.g., RCS).*
2. *Centralized VCS (CVCS) – A single server stores all versions, and clients check out files (e.g., SVN, CVS).*
3. *Distributed VCS (DVCS) – Every developer has a complete copy of the repository (e.g., Git, Mercurial).*

Popular VCS Tools for SCM:

- *Git (DVCS) – Widely used in open source and DevOps.*
- *Apache Subversion (SVN) – Centralized system for enterprise projects.*
- *Mercurial – Similar to Git but simpler command set.*
- *Perforce – Often used in game development and large binary file storage.*

Git is Preferred in DevOps:

- *Distributed architecture enables faster collaboration.*
- *Lightweight branching and merging makes feature development easier.*
- *Open-source and widely adopted in CI/CD pipelines.*
- *Offline capability allows work without constant internet.*
- *Integrates seamlessly with GitHub, GitLab, Bitbucket.*

Comparison between Tools:

Feature	*Git (DVCS)*	*SVN (CVCS)*	*Mercurial (DVCS)*
Storage Model	*Distributed*	*Centralized*	*Distributed*
Speed	*Very fast*	*Slower*	*Fast*
Offline Work	*Yes*	*No*	*Yes*
Popularity	*Very High*	*Medium*	*Low*
Branching/Merging	*Lightweight & Easy*	*Heavy & Complex*	*Easy*

Best Practices in VCS

1. *Commit frequently with clear messages.*
2. *Use branches for new features or bug fixes.*
3. *Pull latest changes before starting new work.*
4. *Avoid committing sensitive data (use .gitignore).*
5. *Keep repository organized and remove unused branches.*

Conclusion:

Version Control Systems like Git are essential in modern software development for tracking changes, enabling collaboration, and ensuring code quality. In the DevOps workflow, Git plays a central role in managing source code, enabling automation in CI/CD pipelines, and supporting collaborative development.